Centro de Educação Superior de Brasília
Centro Universitário Instituto de Educação Superior de Brasília

Professor: Rodrigo Gonçalves Pinto
Disciplina: Programação Paralela

Relatório — Coeficiente de Variação com OpenMP

Aluno: Alanna Tomaz de Aquino Martins **Matrícula:** 2412130055

1. Introdução

Neste trabalho foi calculado o coeficiente de variação (CV) de duas variáveis de uma base de 1000 pessoas: estatura (em cm) e peso (em kg). Para isso, os dados precisam ser agrupados em intervalos de classe (8 cm para estatura e 4 kg para peso) antes de calcular média, desvio padrão e CV.

O programa foi escrito em C++ com duas versões: uma sequencial e uma com OpenMP, para poder comparar o tempo de cada uma depois.

A base de dados foi gerada com um programa separado (data/gerar_dados.cpp), usando uma seed fixa para sempre gerar os mesmos dados. A estatura segue uma distribuição normal (média 170 cm, desvio 9 cm) e o peso foi calculado a partir de um IMC aleatório aplicado na estatura de cada pessoa.

2. Partes paralelizadas e por quê

O cálculo de cada variável tem 4 passos: achar o mínimo e o máximo, montar o histograma de frequência por classe, calcular a média ponderada e calcular o desvio padrão. Paralelizei os 4:

- **Mínimo e máximo**: usei reduction(min:) e reduction(max:). Foi o mais fácil — cada thread guarda seu próprio mínimo/máximo e no final o OpenMP junta tudo.

- **Histograma**: esse foi o mais complicado de paralelizar. Não dá pra usar reduction direto no vetor de frequências porque mais de uma thread pode tentar incrementar a mesma posição ao mesmo tempo e dar resultado errado (condição de corrida). Resolvi isso dando pra cada thread um histograma próprio (local), que ela preenche sem disputar com ninguém, e só no final cada thread soma o histograma dela no histograma de verdade, dentro de um #pragma omp critical.

- **Média e desvio padrão**: usei reduction(+:) nas duas, que é o jeito mais simples de somar valores em paralelo — cada thread soma sua parte e o OpenMP soma tudo no final.

Pra ter certeza que estava tudo certo, comparei o resultado da versão sequencial com o da paralela pras duas variáveis, e os valores de média, desvio padrão e CV deram exatamente iguais.

3. Resultados obtidos

Variável	Classes (k)	Largura	Média (μ)	Desvio padrão (σ)	CV
Estatura (cm)	7	8 cm	170.18	8.95	5.26 %
Peso (kg)	21	4 kg	67.15	12.31	18.33 %

(valores obtidos com data/dataset.txt, gerado com a seed 20251111; dá pra reproduzir rodando ./coeficiente_variacao)

O CV do peso ficou bem maior que o da estatura, ou seja, o peso varia proporcionalmente mais entre as 1000 pessoas do que a estatura. Isso faz sentido porque o peso costuma variar mais de pessoa para pessoa do que a altura.

4. Análise de desempenho

Para conseguir medir o tempo direito, rodei o cálculo completo (min/max + histograma + estatísticas) 2000 vezes seguidas, porque com só 1000 dados uma execução só demora microssegundos — tempo curto demais pra medir com precisão.

Variável	Tempo sequencial	Tempo paralelo	Speedup
Estatura (cm)	0.0130 s	2.8950 s	0.00x
Peso (kg)	0.0140 s	3.3900 s	0.00x

O resultado foi surpreendente no começo: a versão paralela ficou **bem mais lenta** que a sequencial. Mas pesquisando entendi que isso faz sentido: toda vez que o programa entra num #pragma omp parallel, o OpenMP precisa criar as threads, dividir o trabalho e depois juntar tudo de novo, e isso tem um custo fixo. Com só 1000 elementos e no máximo 21 classes, esse custo de criar/sincronizar as 20 threads da minha máquina é muito maior do que o próprio cálculo, que é bem rápido.

Ou seja: o paralelismo está funcionando certo (os resultados batem com a versão sequencial), só que pra esse tamanho de dado (N=1000) ele não vale a pena na prática — a versão sequencial seria mais rápida nesse caso. Mesmo assim, implementei e medi a versão paralela porque o objetivo do exercício é aprender a usar o OpenMP, não necessariamente ganhar tempo com uma

base tão pequena.

5. Conclusão

Consegui calcular o coeficiente de variação da estatura e do peso a partir dos dados agrupados em classes, e as versões sequencial e paralela dão o mesmo resultado. Sobre o desempenho, aprendi que paralelizar tem um custo fixo (criar e sincronizar as threads) que só compensa quando o trabalho é grande o suficiente — com os 1000 dados do exercício esse custo é maior que o ganho, então a versão paralela acabou ficando mais lenta que a sequencial.